SCRUM-199 Spike: Django + WebRTC and UDP/TCP telemetry

Nov 11, 2026 · @kokos

Summary

Run all real-time networking in one asyncio service called `realtime`, next to the Django web app. It holds the aiortc peers, the UDP listener and the board TCP gateway. Django stays a normal web app and never touches packets.

- **WebRTC:** keep HTTP POST signalling (SCRUM-79). Django Channels is not needed, because aiortc sends all its candidates in the answer and the data channels carry all server-to-client pushes. The `/rtc/offer` route goes to the `realtime` service, not Django.
- **Data channels:** the client opens a `stability` channel with `ordered: false, maxRetransmits: 0` and keeps the existing reliable channel for transcript and events. Use aiortc 1.16 or later, which fixed SCTP partial-reliability bugs.
- **UDP:** an asyncio `DatagramProtocol` inside `realtime`. A packet goes from socket to data channel in the same process, so there is no broker hop in the 50 ms budget.
- **TCP:** the board keeps one TLS connection to `realtime` for the whole session. It carries batches up, and acks, session start/stop and clock sync down. It also gives online status and last seen time (SCRUM-151).
- **DB:** `realtime` writes telemetry to TimescaleDB with psycopg 3 (async pool). It reads devices and sessions through the Django ORM sparingly and caches them in memory. Django tells `realtime` about session events with Postgres `LISTEN/NOTIFY`, so no new container is needed.
- **Packet format:** binary, little-endian, fixed size: a 12-byte header, 36-byte samples and an 8-byte HMAC tag. One UDP live packet is 56 bytes. The same sample struct is used in TCP batches (SCRUM-148).
- **Session matching:** the board never knows the session. The server maps `device_id` to the running session (only one per board), and maps sample time to the talk by talk start and end times.
- **Latency:** 50 ms board-to-screen over a public VPS is tight. Wi-Fi power save must be off on the ESP32 and the VPS must be close, and it still has to be measured. 250 ms (SCRUM-152) is safe.

Context and constraints

The design has to fit what Sprint 2 is already building and what Sprint 3 promises. The constraints below come from the Jira backlog and from team answers on 11 Nov.

Constraint	Source
Board streams 50–100 Hz; fields: timestamp, sequence, device ID, 4 load-cell quadrants, IMU roll and pitch, actuator state, mode, version. Same payload for UDP and TCP	SCRUM-148
BLE only for setup: Wi-Fi credentials, server address, device token	SCRUM-147
UDP for live data, periodic TCP batches for the full record, resend after a Wi-Fi drop, no duplicates, gaps flagged	SCRUM-152, SCRUM-153
Unknown devices rejected; one running session per board; online status and last seen	SCRUM-151
Telemetry in a time-series DB in Docker Compose, linked by session, talk and device ID, no names or emails	SCRUM-154
Rolling stability score sent live on the data channel	SCRUM-155
aiortc backend peer, HTTP POST signalling, data channel messages are type plus payload, STUN/TURN configured	SCRUM-64, SCRUM-78/79/80
Stability on an unordered channel with no retransmits; transcript reliable and ordered; messages about every 10 ms	SCRUM-198
Target 50 ms sensor-to-screen (chosen over the 250 ms in SCRUM-152)	team answer
Board MCU is an ESP32 programmed with Arduino	team answer
Server is a public VPS; boards reach it over the internet	team answer
aiortc peer is not built yet, so the process layout is open	team answer

The public VPS matters most for security: UDP and TCP ports are open to the internet, so every packet must be authenticated. It also adds internet latency to the 50 ms budget.

Q1: WebRTC alongside Django

Run aiortc in its own long-lived asyncio process, not inside Django's request workers. Keep HTTP POST signalling and skip Django Channels.

Why not inside Django. Django can run async views under ASGI, but every `RTCPeerConnection` lives in one event loop for the whole session. With several uvicorn workers, an offer, a session start and a board packet can each land in a different worker. One process that owns every peer avoids this. Django's own maintainers also advise against long-running servers inside Django processes (Django ticket #35583).

Why no Django Channels. Channels adds WebSockets and a Redis channel layer. We don't need either:

- Signalling is one request: the client POSTs an offer and gets an answer. aiortc gathers all its ICE candidates before it returns the answer, so there is nothing to trickle afterwards.
- Every push from the server (transcript, invites, analysis results, stability) already goes over data channels (SCRUM-64, SCRUM-65).

Signalling route. A reverse proxy (nginx or Caddy) sends `POST /rtc/offer` to `realtime`. `realtime` checks the user's Django session cookie or token by reading Django's session table, then answers. Django serves everything else.

Data channels. The client creates its channels before `createOffer`, and aiortc receives them through `pc.on("datachannel")`:

Channel	Options	Carries
`events` (existing)	reliable, ordered (default)	transcript, invites, analysis results, warnings
`stability`	`ordered: false, maxRetransmits: 0`	live stability messages, about every 10 ms

aiortc has honoured `maxRetransmits` and `maxPacketLifeTime` since 0.9.17. Release 1.16.0 fixed several partial-reliability bugs that could lose messages or stall a stream, so pin `aiortc>=1.16` (changelog).

The stability message keeps the SCRUM-80 format, type plus payload, as compact JSON:

```
{"type":"stability","v":1,"seq":48213,"t":1762862400123,"cop":
[0.12,-0.03],"roll":1.4,"pitch":-0.6,"score":82}
```

That is about 110 bytes, or 11 KB/s per viewer at 100 Hz. The client drops any message with a `seq` lower than the last one it drew, and redraws on `requestAnimationFrame` (SCRUM-198).

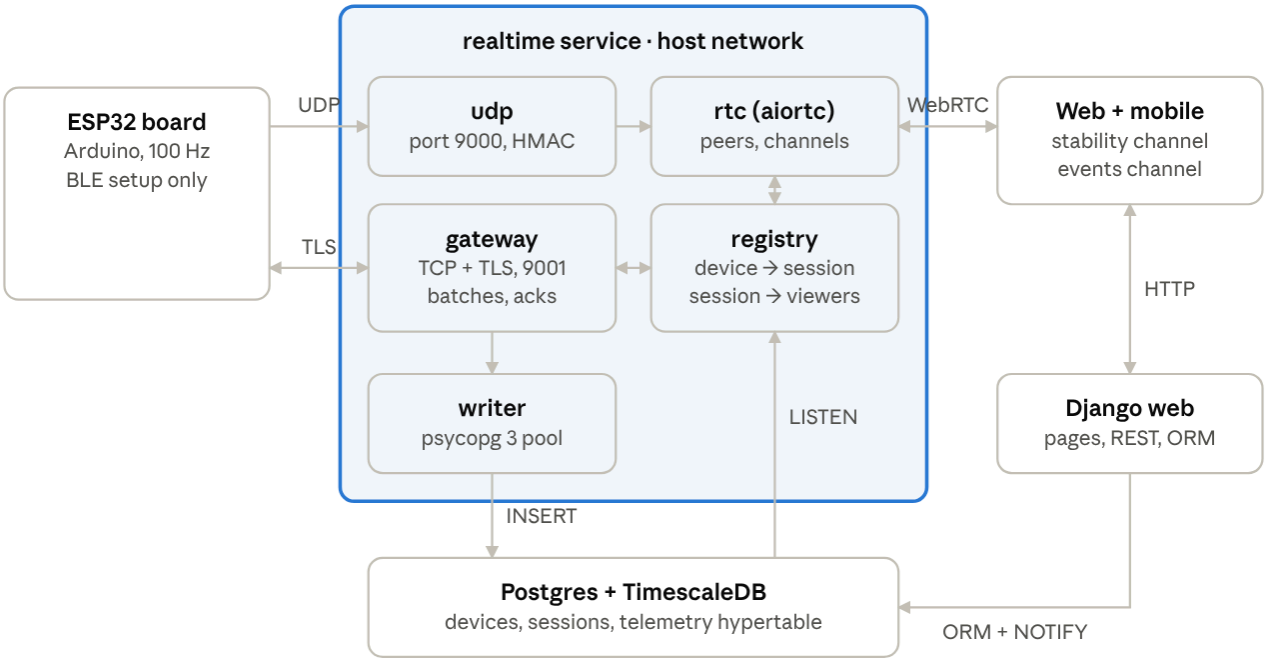
Docker networking. aioice picks random local UDP ports for ICE, and you can't limit them to a range yet (aioice #47 and PR #63 are still open). Under Docker's bridge network, aiortc would also advertise the container's private IP. So run `realtime` with `network_mode: host` on the VPS, and it advertises the VPS public IP directly. Add coturn with TURN over TCP/TLS on 443 for networks that block UDP (eduroam, some corporate Wi-Fi). aiortc has no ICE-TCP, so TURN is the only fallback.

CPU. aiortc's SCTP and DTLS are pure Python. Whisper must not run on the same event loop. Run faster-whisper in a separate process (a `ProcessPoolExecutor` or its own worker) so a long transcription can't delay stability packets.

Q2: UDP and TCP listeners, process layout and DB access

Put the UDP listener and the TCP gateway in the same `realtime` process as the aiortc peers. Give it its own container from the same Docker image as Django, started with `python -m realtime` instead of uvicorn.

One realtime process owns every live path; Django never touches packets



telemetry architecture · realtime service, Django, DB, board, clients

Board packets enter `realtime` and leave on data channels without crossing a process boundary. Django only changes sessions and announces them through Postgres.

Why one process. A live packet has to reach the viewer's data channel within milliseconds. In one process, forwarding is a dictionary lookup and a `channel.send()`. Split into two processes, every packet also crosses Redis or a socket, and you add a container to run. The TCP gateway sits in the same process because it pushes session start and stop to the board, and the session state already lives there.

Inside `realtime`:

1. `rtc` – aiohttp app for `POST /rtc/offer`, holding every peer connection and its data channels.
2. `udp` – `loop.create_datagram_endpoint()` on one port (for example 9000/udp). It checks the HMAC, drops stale `seq`, computes centre of pressure (CoP) and the rolling score, and forwards to that session's viewers.
3. `gateway` – `asyncio.start_server()` with TLS on one port (for example 9001/tcp) for each board's persistent connection.
4. `registry` – in-memory maps: `device_id → device key + active session`, and `session_id → viewers + current talk`. Loaded at start-up and kept current by `LISTEN/NOTIFY`.
5. `writer` – queues validated batches and inserts them into TimescaleDB.

How it reaches the DB.

- Telemetry writes go through psycopg 3 with an async connection pool, not the ORM. Batches are bulk inserts of many rows, and the ORM's per-object overhead and stale-connection handling in long-running processes don't suit that.
- Device and session lookups use the Django ORM's async methods (`await Device.objects.aget(...)`) after `django.setup()`. This keeps one source of truth for models and migrations. These lookups only run on connect and on session events, never per packet.
- Django sends events with `NOTIFY realtime, '{"event":"session_started","session":...}'` in the same transaction that changes the session. `realtime` keeps one `LISTEN` connection. Invites to the mobile presence channel (SCRUM-65) use the same path.

Board TCP protocol (frames, each with a 4-byte length prefix):

Direction	Frame	Purpose
board → server	`HELLO`	device ID, boot ID, firmware version, HMAC over a server nonce. The token itself is never sent

Direction	Frame	Purpose
server → board	`STATE`	`stream_on` or `stream_off`, batch interval (SCRUM-153), sample rate
board → server	`BATCH`	header + N samples, same sample struct as UDP
server → board	`ACK`	highest contiguous `seq` stored; the board frees its buffer up to that point
both	`PING` / `PONG`	four timestamps for clock offset (see Q3), and a keep-alive

After a Wi-Fi drop the board reconnects, sends `HELLO`, and resends everything after the last `ACK`. That covers the "send the missing data once back online" criterion with no extra logic.

Security. The VPS is public, so the TCP gateway uses TLS (`WiFiClientSecure` on the ESP32, with the server CA pinned). UDP live packets carry a truncated HMAC-SHA256 keyed by the device token from BLE pairing, and the ESP32 has hardware SHA. Packets from unknown devices, with bad tags or with old `(boot_id, seq)` are dropped silently.

Scaling later (SCRUM-140). One `realtime` process should handle the handful of concurrent sessions we expect. If the load test shows it is CPU bound, run several `realtime` instances, each owning a set of boards and sessions, and keep the in-process forwarding.

Q3: Packet format and session matching

Use a fixed-size binary format, and match packets to sessions on the server only. This answers the open question in SCRUM-148: binary, with a documented byte table and a decoder script so the format stays easy to read while debugging.

JSON vs binary for this board:

	Binary struct	JSON
Size of 1 sample	36 B	about 220 B
Wi-Fi drop buffer, 1 min at 100 Hz	216 KB	about 1.3 MB
Encode on ESP32	`memcpy` into a packed struct	ArduinoJson, allocations, float formatting

	Binary struct	JSON
Decode on server	`struct.unpack_from` / NumPy, fixed offsets	`json.loads` per sample
Validation	length and version check, fixed ranges	schema check per field
Readability	needs a decoder script	readable as is

Bandwidth is not the deciding factor: even JSON at 100 Hz is only about 22 KB/s. The real constraint is the board's buffer. The ESP32's internal RAM can't hold even a minute of JSON next to Wi-Fi and TLS.

Layout (little-endian, version 1):

Part	Field	Type	Notes
Header (12 B)	`version`	u8	1
	`msg_type`	u8	1 = live, 2 = batch
	`count`	u16	samples in this message (1 for live)
	`device_id`	u32	numeric ID assigned at registration
	`boot_id`	u32	random at each boot, so `seq` restarting at 0 is not a duplicate
Sample (36 B)	`t_us`	u64	board clock, microseconds since the Unix epoch (SNTP-synced)
	`seq`	u32	+1 per sample, fixed rate
	`lc[4]`	4 × f32	load-cell quadrants; unit to be signed off with mechatronics
	`roll`, `pitch`	2 × i16	0.01° steps (±327°)
	`actuator`	u8	actuator state code
	`mode`	u8	0 = passive, 1 = active
	`flags`	u16	e.g. sensor fault, saturated cell
Tag (8 B)	`hmac`	8 B	HMAC-SHA256 over header + samples, truncated

A live UDP packet holds one sample: 12 + 36 + 8 = **56 bytes**, sent every 10 ms at 100 Hz. Don't bundle several samples into one UDP packet, because each one waits for the next and eats into the 50 ms budget. A TCP batch is the same header with `count` up to 65,535, and TLS already protects it, but the tag is kept so one decoder handles both.

Matching packets to sessions. The board only knows its own `device_id`. The server does the matching:

1. The trainer starts a session and picks board X (SCRUM-139, SCRUM-176). Django saves it and sends `NOTIFY`.
2. `realtime` sets `device X → session S` in its registry and sends `STATE stream_on` to the board over TCP.
3. A UDP packet arrives. `realtime` looks up X's key, checks the tag, finds S, and forwards to S's viewers. No DB call.
4. A batch arrives. Each sample is assigned to the session whose time window contains its `t_us`, not the session running when it arrives. A late resend after a Wi-Fi drop still lands in the right session. Ended sessions stay in the registry for a grace period, for example 10 minutes.
5. Talks come from time too: a sample belongs to the talk whose `started_at`–`ended_at` window contains it. Store `session_id` and `device_id` on each row and join to talks by time range. The trainer may end a talk after its samples are already stored.

Duplicates and gaps (SCRUM-153). Rows go in with `INSERT ... ON CONFLICT DO NOTHING` on a unique index over `(device_id, boot_id, seq, ts)`. TimescaleDB requires every unique index on a hypertable to include the time column (docs). The writer tracks contiguous `seq` ranges per `(session, device, boot)`. When a session ends, the missing ranges are written to the talk as a gap flag.

Clocks. Talk boundaries use server time, but samples use board time. SNTP on the ESP32 is only accurate to roughly 100 ms (makerguides). That's fine for picking a talk but too coarse to measure a 50 ms latency. So the TCP `PING/PONG` frames do an NTP-style exchange of four timestamps. The server keeps the offset from the lowest-RTT ping of the last N, and uses it to correct `t_us` on ingest and for latency logging. The offset error is then about half the RTT asymmetry, typically a few ms.

Latency budget for 50 ms

Over a public VPS, 50 ms is reachable only with Wi-Fi power save off and a nearby server, and it has to be measured to be sure. The data goes from the board up to the VPS and back down to a screen in the same room, so it crosses the internet twice.

Step	Estimate	What controls it
Wait for the next sample	0–10 ms	100 Hz rate; no bundling on UDP
ESP32 Wi-Fi uplink	1–5 ms with power save off; up to ~300 ms with it on	`WiFi.setSleep(false)` after connecting
Board → VPS (internet)	10–30 ms	VPS region; measure from the SDU network
`realtime` processing	under 1 ms	stay on one event loop; Whisper in another process
VPS → client (WebRTC)	10–30 ms	same as above; TURN relay adds a hop
Wait for the next frame	0–17 ms	`requestAnimationFrame` at 60 Hz
Total	**about 25–90 ms**	

These are estimates to check, not measurements. The power-save line is the one that breaks the budget on its own. By default the ESP32 uses modem sleep, so incoming and outgoing data can wait for a beacon interval: up to roughly 300 ms with a listen interval of 3 (esp32.com, Espressif docs).

How to measure (SCRUM-152 criterion):

- Board → server: `server_receive_time − corrected t_us` for every live packet, logged as p50/p95 per session.
- Server → client: once a second the client echoes the `seq` it just drew on the `events` channel. The server halves the round trip.
- End to end = the two added together. The client shows the warning above 500 ms or when packets stop.

If the two internet legs together measure over about 30 ms from SDU, the 50 ms goal can't be met on this setup. In that case, record 50 ms as the goal and 250 ms (SCRUM-152) as the hard limit in the report.

Risks

Risk	Mitigation
One `realtime` process is a single point of failure; a restart drops every live peer	Clients reconnect on their own (SCRUM-64); boards reconnect and resend after the last `ACK`, so no data is lost
aiortc is pure Python, so CPU use per session is unknown	Measure in the SCRUM-140 load test; split boards across several `realtime` instances if needed
No ICE port range in aioice	`network_mode: host` for `realtime`; firewall opens the UDP range in use
Venue Wi-Fi blocks outbound UDP	Live view stops but TCP batches still store everything; the client shows "live connection lost". Fallback: live samples over the TCP connection
ESP32 modem sleep left on	Firmware sets `WiFi.setSleep(false)`; the board reports the setting in `HELLO`

Open questions

- [ ] Does the ESP32 module have PSRAM? Internal RAM holds well under a minute of samples next to Wi-Fi and TLS. 4 MB of PSRAM holds about 19 minutes at 100 Hz. Without it, buffer to flash (LittleFS ring buffer).
- [ ] Load-cell unit and range, and the final rate (50 or 100 Hz): to sign off with mechatronics in SCRUM-148.
- [ ] Which VPS provider and region? Pick it by measured RTT from SDU.
- [ ] Is Redis already in the stack (for example Celery for voice analysis)? If so, Redis pub/sub can replace `LISTEN/NOTIFY`; otherwise keep Postgres only.

Impact on tickets

Ticket	Change from this spike
SCRUM-78, SCRUM-79	Build the aiortc peer and `/rtc/offer` in the `realtime` service, not in Django views

Ticket	Change from this spike
SCRUM-64	coturn with TURN over TCP/TLS on 443; `realtime` on host networking
SCRUM-148	Adopt binary format v1 (Q3 layout) plus a Python decoder script in the repo
SCRUM-151	The device token is also the HMAC key; online status and last seen come from the TCP gateway
SCRUM-177, SCRUM-181	ESP32: power save off, `WiFiClientSecure` with a pinned CA, HMAC via mbedTLS, random `boot_id`
SCRUM-178, SCRUM-179	UDP listener and registry in `realtime`; `stability` channel unordered, no retransmits
SCRUM-183	TCP gateway frames: `HELLO`, `STATE`, `BATCH`, `ACK`, `PING/PONG`
SCRUM-187	Hypertable with `session_id`, `device_id`, `boot_id`, `seq`, `ts`; unique index includes `ts`
SCRUM-158	Simulator speaks the same protocol; worth moving up so Sprint 3 can be tested without hardware

Sources

- aiortc changelog: partial reliability since 0.9.17, SCTP fixes in 1.16.0
- aiortc 1.15.0 release record: July 2026, Python 3.10+
- aioice issues: port range issue #47 and PR #63 still open
- aiortc trickle ICE discussion
- MDN: createDataChannel
- Django asynchronous support and ticket #35583
- TimescaleDB: hypertables and unique indexes
- Espressif: Wi-Fi performance and power save and esp32.com thread on modem sleep delay
- Makerguides: ESP32 SNTP accuracy
- Jira: SCRUM-64, 78–80, 139, 140, 147–158, 177–187, 198